`04-Express-API/01-rest-and-express.md`

🌐 Introduction to REST & Express

🎯 What is a RESTful API?

REST = Representational State Transfer

A standard architectural style for building web services

📖 RESTful Services

Key Principles

REST is an architectural style for building HTTP services:

- Uses standard HTTP methods (GET, POST, PUT, DELETE)
- Resources are identified by URLs
- Stateless communication
- Client-server architecture
- JSON data format (usually)

Benefits

✅ Simple and standardized
✅ Platform independent
✅ Scalable
✅ Easy to understand and use

🔑 HTTP Methods (CRUD Operations)

HTTP Method	CRUD	Operation	Example
GET	Read	Retrieve data	GET /api/courses
POST	Create	Add new resource	POST /api/courses
PUT	Update	Update entire resource	PUT /api/courses/1
PATCH	Update	Partial update	PATCH /api/courses/1
DELETE	Delete	Remove resource	DELETE /api/courses/1

⚡ What is Express?

Fast, Minimalist Web Framework

Express is the most popular web framework for Node.js:

- 🚀 Fast and lightweight
- 🛤️ Powerful routing
- 🔌 Middleware support
- 📦 Large ecosystem
- 🎯 Easy to learn

Why Express?

- Built on top of Node's HTTP module
- Much simpler than raw Node.js
- Industry standard
- Great documentation
- Active community

📦 Installing Express

Create a New Project

```
# Create project directory
mkdir express-demo
cd express-demo

# Initialize npm
npm init -y

# Install Express
npm i express
```

package.json will now include:

```
{
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

🏗️ First Express Application

Create index.js

```
const express = require('express');
const app = express();

// Define a route
app.get('/', (req, res) => {
    res.send('Hello World');
});
```

```
// Start the server
app.listen(3000, () => {
    console.log('Listening on port 3000...');
});
```

Run it!

```
node index.js
```

Output:

```
Listening on port 3000...
```

Open browser: `http://localhost:3000/`

🔍 Understanding the Code

Line by Line

```
const express = require('express');
```

Import Express module

```
const app = express();
```

Create Express application

```
app.get('/', (req, res) => {
    res.send('Hello World');
});
```

Define GET route for home page

```
app.listen(3000, () => {
```

```
    console.log('Listening on port 3000...');
});
```

Start server on port 3000

📨 Request & Response Objects

📥 Request (req)

Contains information from the client:

- 🌐 **req.params** - Route parameters
  `/api/users/:id` → `req.params.id`
- 🔍 **req.query** - Query strings
  `/search?q=node` → `req.query.q`
- 📦 **req.body** - Request body (POST data)
  `{ "name": "John" }`
- 🏷️ **req.headers** - HTTP headers
  `Content-Type`, `Authorization`

📤 Response (res)

Methods to send data back to client:

- ✉️ **res.send()** - Send any type
  `res.send('Hello')` or `res.send({ name: 'John' })`
- 📋 **res.json()** - Send JSON explicitly
  `res.json({ success: true })`
- 🔢 **res.status()** - Set status code
  `res.status(404).send('Not Found')`

- 🔗 **res.redirect()** - Redirect to URL
  `res.redirect('/home')`

Every route handler receives these two objects:
`app.get('/path', (req, res) => { ... })`

🛤️ Creating API Endpoints

Example: Courses API

```
const express = require('express');
const app = express();

const courses = [
    { id: 1, name: 'course1' },
    { id: 2, name: 'course2' },
    { id: 3, name: 'course3' }
];

// GET all courses
app.get('/api/courses', (req, res) => {
    res.send(courses);
});

app.listen(3000, () => {
    console.log('Listening on port 3000...');
});
```

Test it: `http://localhost:3000/api/courses`

Output: `[{"id":1,"name":"course1"},{"id":2,"name":"course2"},{"id":3,"name":"course3"}]`

🎨 Response Types

Different Ways to Respond

```
// Send plain text
res.send('Hello World');

// Send JSON
res.send({ message: 'Hello' });

// Send array
res.send([1, 2, 3]);

// Send HTML
res.send('<h1>Hello World</h1>');

// Send status code
res.status(404).send('Not Found');

// JSON explicitly
res.json({ message: 'Hello' });
```

Express automatically sets the correct `Content-Type` header!

🔄 No More If Blocks!

Before Express (Raw Node.js)

```
const server = http.createServer((req, res) => {
    if (req.url === '/') {
        res.write('Hello World');
        res.end();
    }
    if (req.url === '/api/courses') {
        res.write(JSON.stringify([1, 2, 3]));
        res.end();
    }
});
```

With Express

```
app.get('/', (req, res) => {
    res.send('Hello World');
});

app.get('/api/courses', (req, res) => {
    res.send([1, 2, 3]);
});
```

✅ Much cleaner and easier to read!

🎯 Key Takeaways

REST & Express Basics

✅ **REST** uses standard HTTP methods for CRUD operations
✅ **Express** simplifies building web APIs in Node.js
✅ **Routes** define endpoints (URLs) for your API
✅ **req** object contains request data
✅ **res** object is used to send responses
✅ Express handles **content-type** headers automatically

🏠 Course Home | 📘 Chapter 4 Home

← Previous: Chapter 4 Intro | Next: GET Requests →